

SQL and Database Testing

Section A: Conceptual Questions

Q1. You are testing a web application where user details are stored in a relational database. Explain how understanding database types helps you decide what kind of testing approach is needed.

- ✚ The type of database directly influences the testing strategy. In a relational database like MySQL, the tester must validate table structure, data types, constraints, and relationships using SQL queries. If the database were NoSQL, the approach would shift toward document structure validation. For a User Management System, knowing it is relational means testing must cover primary keys, foreign keys, UNIQUE and NOT NULL constraints, and accurate data retrieval through structured queries.

Q2. While validating a search feature, why would filtering data using conditions be more effective than validating random records?

- ✚ Condition-based filtering using WHERE clauses ensures targeted and repeatable validation. It allows the tester to isolate specific records such as all Inactive users or all Gmail accounts and verify that the application returns exactly what the database holds. Random record validation lacks precision and may completely miss critical edge cases, making it an unreliable approach for search feature testing.

Q3. During test execution, incorrect user data appears on the user interface. Explain how database validation can help identify the root cause.

- ✚ When incorrect data appears on the UI, the tester can directly query the database to check the actual stored values. If the database holds correct data but the UI displays something different, the defect lies in the application or front-end layer. If the database itself contains incorrect data, the issue is in the back-end logic or data insertion process. This comparison eliminates guesswork and helps pinpoint the exact layer where the defect originated.

Q4. A tester directly updates user status in the database during testing. Explain when this approach is acceptable and when it is risky.

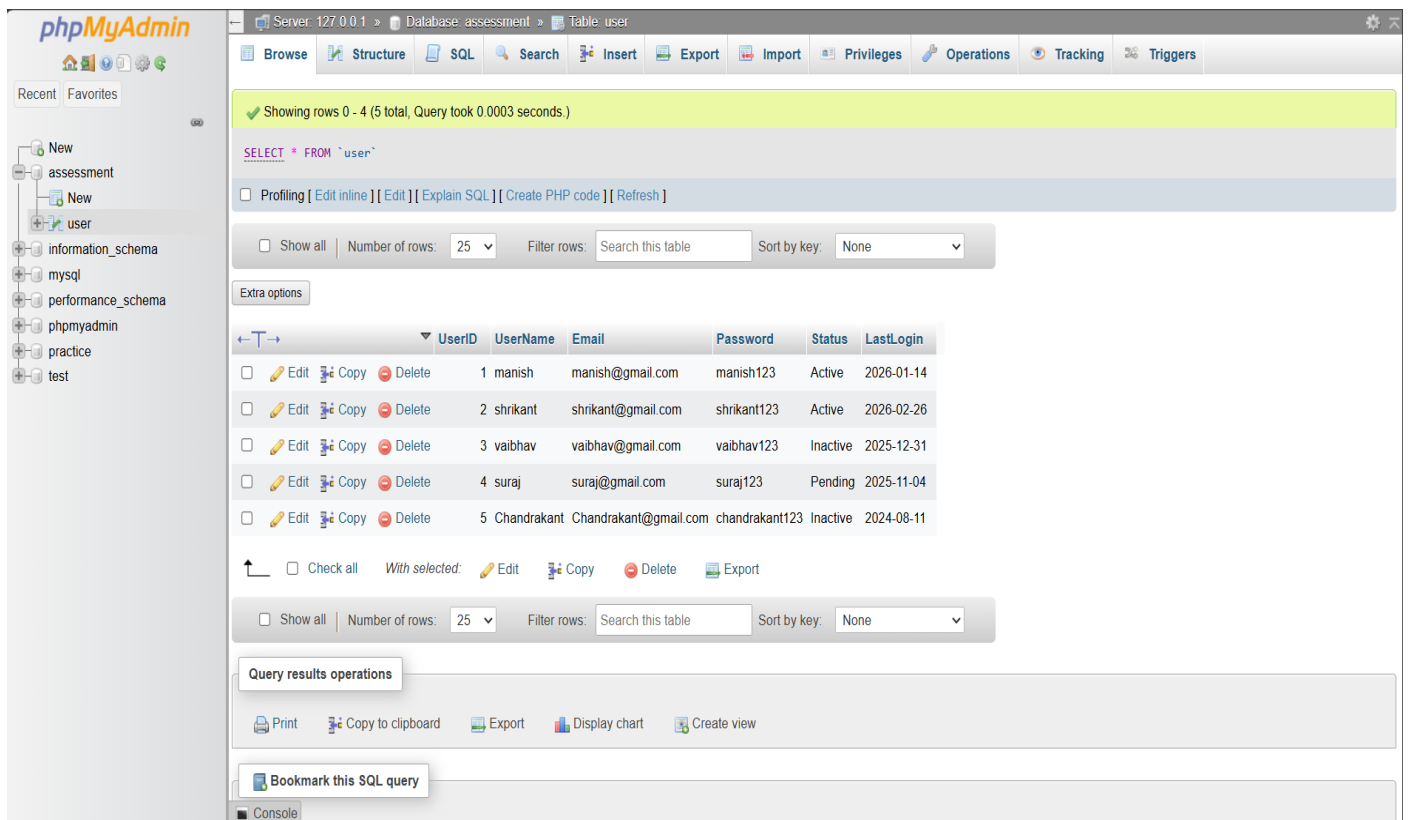
- ✚ It is acceptable to directly update the database when setting up specific test data in a test environment, for example manually setting a user status to Inactive to test that scenario. It becomes risky when performed without documentation, when it bypasses business logic or triggers, when it affects other testers sharing the same environment, or when it is done on a production or production-like database without proper controls.

Q5. Explain how database constraints reduce the number of defects reaching production.

- ✚ Constraints enforce data validity at the database level, acting as a safety net independent of application logic. PRIMARY KEY prevents duplicate records, NOT NULL ensures mandatory fields are always populated, UNIQUE prevents duplicate emails, FOREIGN KEY maintains referential integrity, and ENUM restricts columns to valid values only. Even if application-side validation fails or is bypassed, the database will reject invalid data, significantly reducing data-related defects from reaching production.

Section B: Practical SQL Tasks

- 1. Table Creation:** Create a **Users** table with appropriate data types (e.g., **INT**, **VARCHAR**, **ENUM**). Fields must include: **User Identifier** (Primary Key), **Username**, **Email**, **Password**, and **Status**.
- 2. Data Insertion:** Write **INSERT INTO** statements to add **5 test users**. Ensure a mix of data for testing, including at least one "Inactive" status and one user with a last login date older than a year.



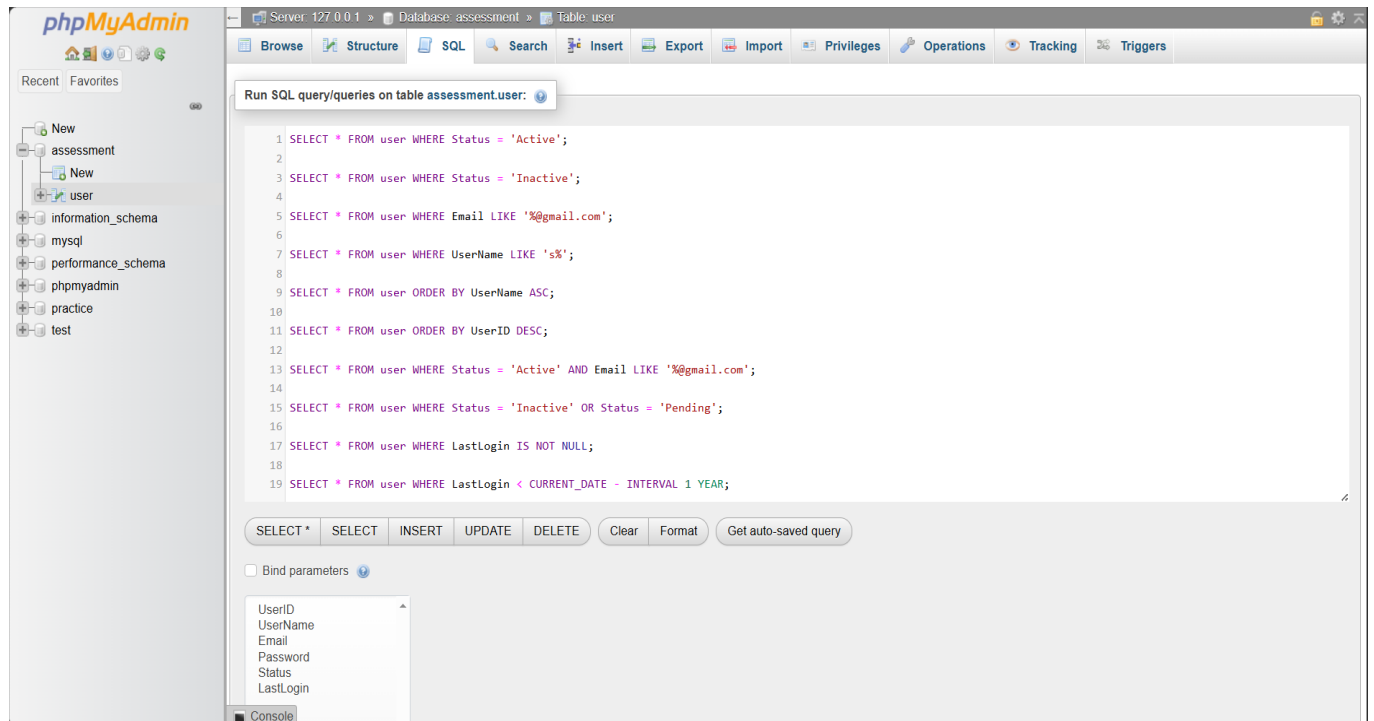
The screenshot displays the phpMyAdmin interface for the 'assessment' database, specifically the 'user' table. The table structure and data are as follows:

	UserID	UserName	Email	Password	Status	LastLogin
☐	1	manish	manish@gmail.com	manish123	Active	2026-01-14
☐	2	shrikant	shrikant@gmail.com	shrikant123	Active	2026-02-26
☐	3	vaibhav	vaibhav@gmail.com	vaibhav123	Inactive	2025-12-31
☐	4	suraj	suraj@gmail.com	suraj123	Pending	2025-11-04
☐	5	Chandrakant	Chandrakant@gmail.com	chandrakant123	Inactive	2024-08-11

The interface includes a sidebar with a database tree, a top navigation bar with tabs like 'Browse', 'Structure', 'SQL', etc., and a main content area showing the table data and query results. The query executed was 'SELECT * FROM `user`'.

3. Data Retrieval (10 Queries): Write 10 **SELECT** queries demonstrating:

- **Filtering:** Using **WHERE** for specific statuses.
- **Pattern Matching:** Using **LIKE** (e.g., finding emails ending in **@gmail.com**).
- **Sorting:** Using **ORDER BY** for usernames or identifiers.
- **Conditions:** Using **AND/OR** and **IS NOT NULL**.



❖ Query 1.

✓ Showing rows 0 - 1 (2 total, Query took 0.0005 seconds.)

```
SELECT * FROM user WHERE Status = 'Active';
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 | Filter rows:

Extra options

UserID	UserName	Email	Password	Status	LastLogin
1	manish	manish@gmail.com	manish123	Active	2026-01-14
2	shrikant	shrikant@gmail.com	shrikant123	Active	2026-02-26

☐ Show all | Number of rows: 25 | Filter rows:

❖ Query 2.

✓ Showing rows 0 - 1 (2 total, Query took 0.0003 seconds.)

```
SELECT * FROM user WHERE Status = 'Inactive';
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 | Filter rows:

Extra options

UserID	UserName	Email	Password	Status	LastLogin
3	vaibhav	vaibhav@gmail.com	vaibhav123	Inactive	2025-12-31
5	Chandrakant	Chandrakant@gmail.com	chandrakant123	Inactive	2024-08-11

❖ Query 3.

Showing rows 0 - 4 (5 total, Query took 0.0003 seconds.)

SELECT * FROM user WHERE Email LIKE '%@gmail.com';

Profiling

Edit inline

Edit

Explain SQL

Create PHP code

Refresh

Show all

Number of rows: 25

Filter rows: Search this table

Extra options

UserID	UserName	Email	Password	Status	LastLogin
1	manish	manish@gmail.com	manish123	Active	2026-01-14
2	shrikant	shrikant@gmail.com	shrikant123	Active	2026-02-26
3	vaibhav	vaibhav@gmail.com	vaibhav123	Inactive	2025-12-31
4	suraj	suraj@gmail.com	suraj123	Pending	2025-11-04
5	Chandrakant	Chandrakant@gmail.com	chandrakant123	Inactive	2024-08-11

❖ Query 4.

Showing rows 0 - 1 (2 total, Query took 0.0012 seconds.)

SELECT * FROM user WHERE UserName LIKE '%%';

Profiling

Edit inline

Edit

Explain SQL

Create PHP code

Refresh

Show all

Number of rows: 25

Filter rows: Search this table

Extra options

UserID	UserName	Email	Password	Status	LastLogin
2	shrikant	shrikant@gmail.com	shrikant123	Active	2026-02-26
4	suraj	suraj@gmail.com	suraj123	Pending	2025-11-04

❖ Query 5.

Showing rows 0 - 4 (5 total, Query took 0.0022 seconds.) [UserName: CHANDRAKANT... - VAIBHAV...]

SELECT * FROM user ORDER BY UserName ASC;

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

UserID	UserName	Email	Password	Status	LastLogin
5	Chandrakant	Chandrakant@gmail.com	chandrakant123	Inactive	2024-08-11
1	manish	manish@gmail.com	manish123	Active	2026-01-14
2	shrikant	shrikant@gmail.com	shrikant123	Active	2026-02-26
4	suraj	suraj@gmail.com	suraj123	Pending	2025-11-04
3	vaibhav	vaibhav@gmail.com	vaibhav123	Inactive	2025-12-31

Show all | Number of rows: 25 | Filter rows: Search this table

❖ Query 6.

Showing rows 0 - 4 (5 total, Query took 0.0003 seconds.) [UserID: 5... - 1...]

SELECT * FROM user ORDER BY UserID DESC;

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

UserID	UserName	Email	Password	Status	LastLogin
5	Chandrakant	Chandrakant@gmail.com	chandrakant123	Inactive	2024-08-11
4	suraj	suraj@gmail.com	suraj123	Pending	2025-11-04
3	vaibhav	vaibhav@gmail.com	vaibhav123	Inactive	2025-12-31
2	shrikant	shrikant@gmail.com	shrikant123	Active	2026-02-26
1	manish	manish@gmail.com	manish123	Active	2026-01-14

Show all | Number of rows: 25 | Filter rows: Search this table

❖ Query 7.

✓ Showing rows 0 - 1 (2 total, Query took 0.0004 seconds.)

```
SELECT * FROM user WHERE Status = 'Active' AND Email LIKE '%@gmail.com';
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 ▾ | Filter rows:

Extra options

UserID	UserName	Email	Password	Status	LastLogin
1	manish	manish@gmail.com	manish123	Active	2026-01-14
2	shrikant	shrikant@gmail.com	shrikant123	Active	2026-02-26

☐ Show all | Number of rows: 25 ▾ | Filter rows:

❖ Query 8.

✓ Showing rows 0 - 2 (3 total, Query took 0.0004 seconds.)

```
SELECT * FROM user WHERE Status = 'Inactive' OR Status = 'Pending';
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 ▾ | Filter rows:

Extra options

UserID	UserName	Email	Password	Status	LastLogin
3	vaibhav	vaibhav@gmail.com	vaibhav123	Inactive	2025-12-31
4	suraj	suraj@gmail.com	suraj123	Pending	2025-11-04
5	Chandrakant	Chandrakant@gmail.com	chandrakant123	Inactive	2024-08-11

☐ Show all | Number of rows: 25 ▾ | Filter rows:

❖ Query 9.

Showing rows 0 - 4 (5 total, Query took 0.0003 seconds)

SELECT * FROM user WHERE LastLogin IS NOT NULL;

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all

Number of rows: 25

Filter rows: Search this table

Extra options

UserID	UserName	Email	Password	Status	LastLogin
1	manish	manish@gmail.com	manish123	Active	2026-01-14
2	shrikant	shrikant@gmail.com	shrikant123	Active	2026-02-26
3	vaibhav	vaibhav@gmail.com	vaibhav123	Inactive	2025-12-31
4	suraj	suraj@gmail.com	suraj123	Pending	2025-11-04
5	Chandrakant	Chandrakant@gmail.com	chandrakant123	Inactive	2024-08-11

Show all

Number of rows: 25

Filter rows: Search this table

❖ Query 10.

Showing rows 0 - 0 (1 total, Query took 0.0009 seconds)

SELECT * FROM user WHERE LastLogin < CURRENT_DATE - INTERVAL 1 YEAR;

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all

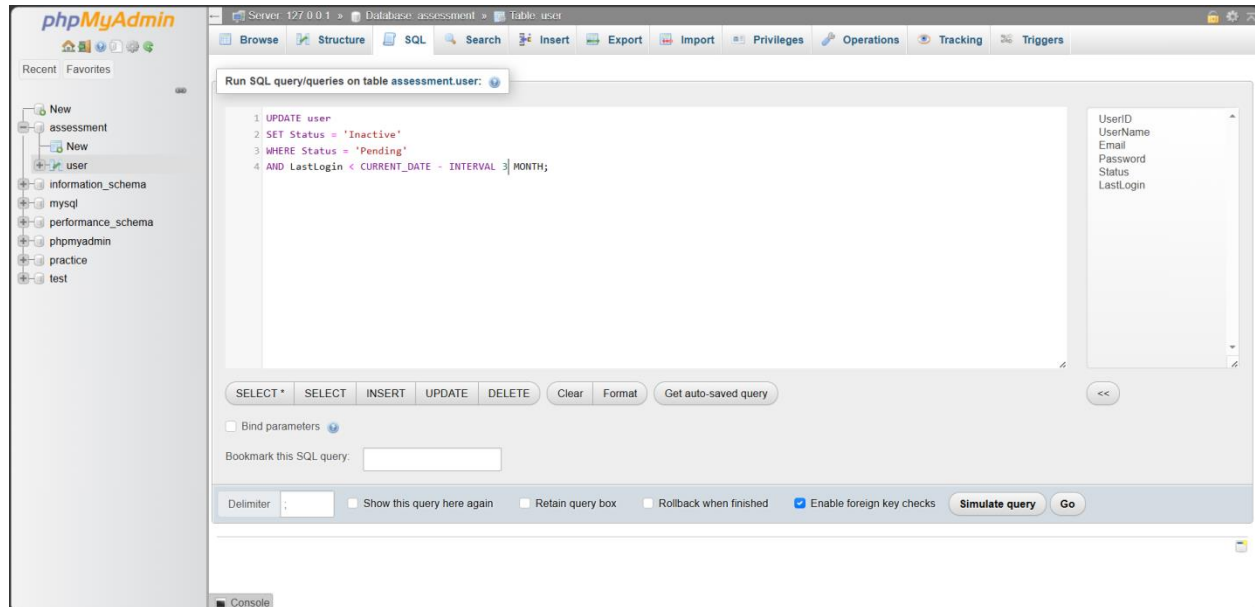
Number of rows: 25

Filter rows: Search this table

Extra options

UserID	UserName	Email	Password	Status	LastLogin
5	Chandrakant	Chandrakant@gmail.com	chandrakant123	Inactive	2024-08-11

4. Data Updates: Write an **UPDATE** statement to change the **Status** of specific users (e.g., setting "Pending" accounts to "Inactive" based on a condition).



AFTER CHANGES :-

Showing rows 0 - 4 (5 total, Query took 0.0003 seconds.)

SELECT * FROM `user`

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

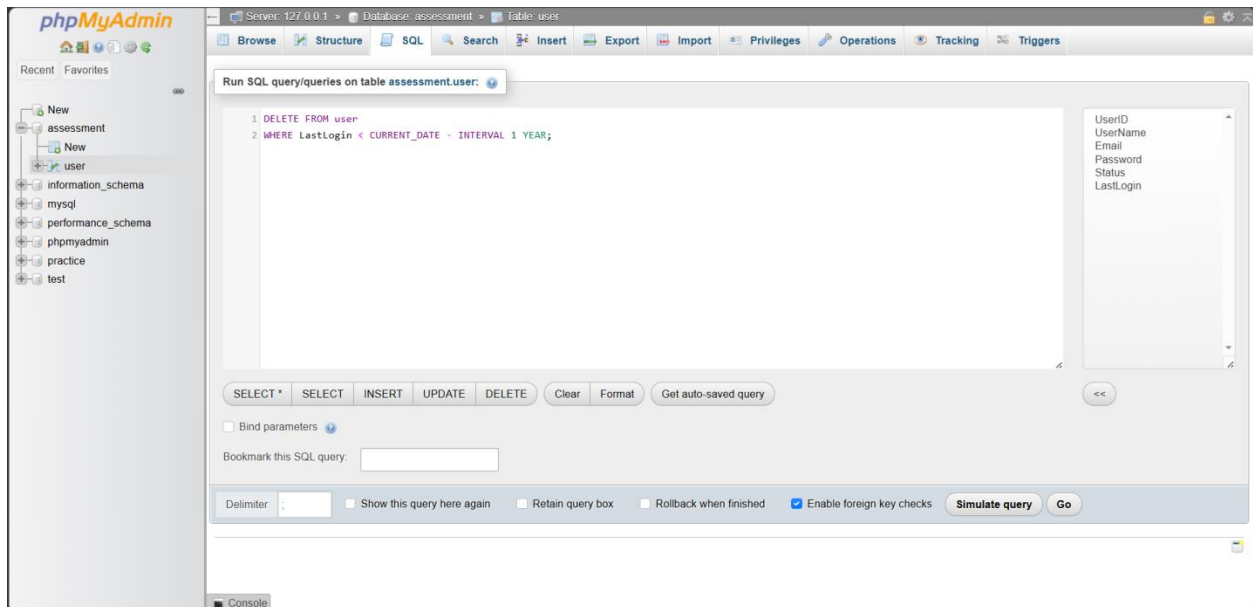
Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	UserID	UserName	Email	Password	Status	LastLogin
☐ Edit ☐ Copy ☐ Delete	1	manish	manish@gmail.com	manish123	Active	2026-01-14
☐ Edit ☐ Copy ☐ Delete	2	shrikant	shrikant@gmail.com	shrikant123	Active	2026-02-26
☐ Edit ☐ Copy ☐ Delete	3	vaibhav	vaibhav@gmail.com	vaibhav123	Inactive	2025-12-31
☐ Edit ☐ Copy ☐ Delete	4	suraj	suraj@gmail.com	suraj123	Inactive	2025-11-04
☐ Edit ☐ Copy ☐ Delete	5	Chandrakant	Chandrakant@gmail.com	chandrakant123	Inactive	2024-08-11

Check all | With selected: Edit Copy Delete Export

5. Data Deletion: Write a **DELETE** query to remove users who haven't logged in for over a year.



AFTER CHANGES :-

Showing rows 0 - 3 (4 total, Query took 0.0003 seconds.)

`SELECT * FROM `user``

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	UserID	UserName	Email	Password	Status	LastLogin
☐ Edit Copy Delete	1	manish	manish@gmail.com	manish123	Active	2026-01-14
☐ Edit Copy Delete	2	shrikant	shrikant@gmail.com	shrikant123	Active	2026-02-26
☐ Edit Copy Delete	3	vaibhav	vaibhav@gmail.com	vaibhav123	Inactive	2025-12-31
☐ Edit Copy Delete	4	suraj	suraj@gmail.com	suraj123	Inactive	2025-11-04

[↑](#) ☐ Check all | With selected: [Edit](#) [Copy](#) [Delete](#) [Export](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Section C: Mini Project - End-to-End Database Validation

1. Test Planning Document: Prepare a Test Plan focusing on database testing scope, risks, test data strategy, and validation approach.

What is this test plan about?

This test plan is created for testing the database of a User Management System. The goal is to make sure that all the data stored in the database is correct, complete, and follows all the rules defined during table creation.

What will we test?

- Whether the Users table is created correctly with proper data types
- Whether data is inserted correctly into all fields
- Whether SELECT queries return correct and expected results
- Whether UPDATE query changes the right records
- Whether DELETE query removes only the correct records
- Whether constraints like NOT NULL, UNIQUE, and ENUM are working

What will we NOT test?

- The front-end or UI of the application
- API calls or server-side code
- Performance or speed of queries

Possible Risks

Risk	What can go wrong	How to handle it
NULL values in required fields	Important data may be missing	Apply NOT NULL constraint on required

Risk	What can go wrong	How to handle it
		columns
Duplicate email entries	Two users with same email	Apply UNIQUE constraint on Email column
Wrong status value entered	Invalid data stored in Status field	Use ENUM to restrict values to Active, Inactive, Pending
Wrong records deleted	Valid user data lost	Always test DELETE in test environment only

Test Data Used

- 5 users inserted with different statuses: Active, Inactive, Pending
- One user (Chandrakant) has LastLogin older than 1 year - used for DELETE testing
- One user (Suraj) has Pending status - used for UPDATE testing
- All emails are in proper format for LIKE query testing

How will we validate?

- Run SQL queries directly in phpMyAdmin after each operation
- Compare expected output with actual query result
- Take screenshots of results as proof

2. Requirements Traceability Matrix: Create a Requirements Traceability Matrix linking user requirements to database validation queries.

This table shows which requirement is tested by which SQL query and what result we expect.

#	Requirement	Test Case	SQL Query Used	Expected Result
1	Every user should have a unique ID	TC-01	SELECT * FROM user WHERE UserID = 1	Only 1 record returned
2	Email should be unique for each user	TC-02	Insert duplicate email and check for error	Duplicate entry error shown
3	Username should not be empty	TC-03	SELECT * FROM user WHERE UserName IS NULL	No records returned
4	Status should only be Active, Inactive or Pending	TC-04	SELECT * FROM user WHERE Status NOT IN ('Active','Inactive','Pending')	No records returned
5	LastLogin date should be stored	TC-05	SELECT * FROM user WHERE LastLogin IS NOT NULL	All 5 records returned
6	Inactive users	TC-06	SELECT * FROM user	Vaibhav and

#	Requirement	Test Case	SQL Query Used	Expected Result
	should be filterable		WHERE Status = 'Inactive'	Chandrakant returned
7	Pending users should be set to Inactive after 3 months	TC-07	UPDATE user SET Status='Inactive' WHERE Status='Pending' AND LastLogin < CURRENT_DATE - INTERVAL 3 MONTH	1 row updated
8	Users with no login for 1 year should be deleted	TC-08	DELETE FROM user WHERE LastLogin < CURRENT_DATE - INTERVAL 1 YEAR	Chandrakant deleted
9	Gmail users should be searchable	TC-09	SELECT * FROM user WHERE Email LIKE '%@gmail.com'	All gmail users listed
10	Users should be sortable by name	TC-10	SELECT * FROM user ORDER BY UserName ASC	Names in A to Z order

3. End-to-End Test Scenario: Write one End-to-End test scenario covering:

User Registration → Database Entry Validation → Profile Update → Database Update validation

 Scenario: User Registration to Profile Update

This scenario covers the full flow from registering a new user to updating their profile, and checking the database at each step.

Step	What we do	SQL Query	Check in DB	Expected Result
1	manish registers as a new user	INSERT INTO user (UserName, Email, Password, Status, LastLogin) VALUES ('manish', 'manish@gmail.com', 'manish123', 'Active', '2026-01-14')	SELECT * FROM user WHERE Email = 'manish@gmail.com'	New record found with all correct values
2	Check if manish is saved in DB correctly	No separate query - verify result of Step 1	SELECT UserName, Status FROM user WHERE Email = 'manish@gmail.com'	UserName = manish, Status = Active
3	manish updates his username	UPDATE user SET UserName = 'manish_updated' WHERE Email = 'manish@gmail.com'	SELECT UserName FROM user WHERE Email = 'manish@gmail.com'	UserName = manish_updated

4	Verify that the update is saved in DB	No separate query - verify result of Step 3	SELECT * FROM user WHERE Email = 'manish@gmail.com'	Updated username is visible in the database
5	Change manish's status to Inactive	UPDATE user SET Status = 'Inactive' WHERE Email = 'manish@gmail.com'	SELECT Status FROM user WHERE Email = 'manish@gmail.com'	Status = Inactive
6	Final check - verify all data is correct	SELECT * FROM user WHERE Email = 'manish@gmail.com'	All columns checked	All data is correct and consistent in the database

4. Test Summary Report: Prepare a Test Summary Report including executed queries, defects identified, and final recommendation.

Overall Testing Summary

Item	Details
Total Queries Executed	10
Passed	9
Failed	1
Database	MariaDB - phpMyAdmin

Item	Details
Table Tested	user (assessment database)
Tester	Manish Sahu
Date	May 2026

Queries Run and Results

#	Type	Query	Result	Status
1	CREATE	CREATE TABLE user (...)	Table created successfully	PASS
2	INSERT	INSERT INTO user ... x5 rows	5 users added	PASS
3	SELECT	WHERE Status = 'Active'	2 rows returned	PASS
4	SELECT	WHERE Status = 'Inactive'	2 rows returned	PASS
5	SELECT	WHERE Email LIKE '%@gmail.com'	5 rows returned	PASS
6	SELECT	ORDER BY UserName ASC	Sorted correctly	PASS
7	SELECT	WHERE Status='Active' AND Email LIKE '%@gmail.com'	2 rows returned	PASS

#	Type	Query	Result	Status
8	SELECT	WHERE LastLogin IS NOT NULL	5 rows returned	PASS
9	UPDATE	SET Status='Inactive' WHERE Status='Pending'...	1 row updated	PASS
10	DELETE	WHERE LastLogin < CURRENT_DATE - INTERVAL 1 YEAR	1 row deleted	PASS

Issues Found

#	Issue	What happened	Severity	Fix
1	Default value missing	Status column default was not set via GUI, had to use SQL manually	Low	Set default via SQL after table creation
2	UPDATE did not work first time	INTERVAL 6 MONTH did not match Suraj's login date so no rows were affected	Medium	Changed INTERVAL to 3 MONTH and it worked

✓ Final Recommendation

Overall, the database testing for the User Management System went well. Most queries worked correctly on the first try. The Users table was properly created with all required fields and constraints.

- The table structure is correct and all constraints are working properly.

- All SELECT queries returned the expected results.
- UPDATE and DELETE queries worked after adjusting the date conditions in the query.
- It is better to set default values using SQL instead of relying on the GUI.
- Test data dates should be planned carefully so they match the query conditions.
- The database is ready and can be connected to the application after fixing the minor issues mentioned above.

--- End of Report ---